The Tiny Language Model

A lecture narrative on the decoder-only transformer

Harrison B. Prosper

PHY 6937, *Machine Learning in Physics*, Florida State University

October 2026

Abstract

This note accompanies the notebook `tutorial_tiny_language_model_ABC.ipynb`. It explains, component by component, the decoder-only transformer built there: a model with 136,169 parameters that learns to write the Taylor series, to $\mathcal{O}(x^6)$, of functions such as $\cosh^3(ax) + \tanh(bx)$, one token at a time. The emphasis is on what each component computes, the shapes of the tensors it acts on, and how the trained network is used autoregressively to generate text.

Contents

1 Overview **2**

2 From text to integers **3**

2.1 Tokens and the vocabulary . . . 3

2.2 One sequence, laid out . . . 3

3 The architecture at a glance **4**

4 The components, one at a time **4**

4.1 Embedding: tokens and positions become points (`Decoder`) . . . 4

4.2 Masks: who may look at whom (`TinyLM.make_mask`) . . . 4

4.3 Attention: the (A, B, C) parametrization (`AttentionHead`, `MultiHeadAttention`) 6

4.4 Feed-forward network (`Feedforward`) . . . 8

4.5 Residual connections and layer normalization (`DecoderLayer`) . . . 8

4.6 The output head (`Decoder.linear`) . . . 8

4.7 Where the parameters live . . . 8

5 Training: 127 predictions from one forward pass **9**

5.1 Aligning predictions with targets (`TLMObjective.forward`) . . . 9

5.2 Which predictions are graded (`TinyLM.make_loss_mask`) . . . 9

5.3 Optimization . . . 9

6 Autoregressive generation **10**
6.1 The algorithm (`greedy_generate`) 10
6.2 What the loop computes 11
6.3 Beam search (`beam_generate`) 11
6.4 Why earlier positions need not be recomputed 13

7 Beyond this model **13**

1 Overview

The tiny language model (TLM) is a decoder-only transformer. Given a prompt such as `cosh(a*x)**3 + tanh(b*x)`, it emits the Taylor series of the function to $\mathcal{O}(x^6)$, one token at a time. With greedy decoding it reproduces, character for character, the correct series for $99.3\% \pm 0.3\%$ of 800 held-out functions.

The whole model can be summarized in one sentence: *integers become points in a 72-dimensional space; three times over, each point looks back at the points before it and is then refined on its own; the final points are turned into a probability distribution over the next token; the chosen token is appended, and the process repeats.* Three ideas carry everything that follows.

Representation. A sequence of L tokens becomes a cloud of L points in $\mathbb{R}^d$, with $d = 72$. Each point encodes both *what* the token is and *where* it sits.

Interaction. Attention updates each point by a weighted average of (linearly transformed) points. The causal mask restricts the average to the point itself and the points before it.

Generation. The network answers only one question: *what is the next token?* Long outputs come from feeding each answer back in as input.

Table 1: Configuration of the model in the notebook.

Quantity	Symbol	Value
Vocabulary size	$\lvert\mathbb{V}\rvert$	40
Sequence length (padded)	$L_{\max}$	128
Embedding dimension (`EMB_DIM`)	d	72
Decoder layers	N	3
Attention heads per layer	H	8
Head dimension, d/H	d_h	9
Feed-forward hidden width (`FF_DIM`)	d_{ff}	128
Dropout probability		0.1
Trainable parameters		136,169
Train / validation / test sequences		12,000 / 156 / 800
Optimizer		Adam, 8×10^5 iterations, batch 32
Learning rate		$1.6 \times 10^{-3} \to 1.0 \times 10^{-4}$, 16 steps

Notation. Tensors carry the batch as their first index, as in the Annotated Transformer. B is the batch size and L the length of the sequence being processed. The notebook calls the embedding dimension both d and h; here it is always d.

2 From text to integers

Every training example is a single row of 128 integers: the prompt and its answer are concatenated, not stored as a pair. This is the defining choice of a decoder-only model. There is no separate encoder for the prompt; the prompt is simply the beginning of the sequence the model learns to continue.

2.1 Tokens and the vocabulary

A regular-expression tokenizer splits each string into tokens, trying longer tokens before shorter ones so that `sinh` is never read as `sin` followed by `h`. The 40 distinct tokens are numbered as in Table 2.

Table 2: The vocabulary $\mathbb{V}$.

Codes	Tokens	Role
0–3	`<pad>`, `<sos>`, `<eos>`, `<sep>`	padding, start, end, prompt/answer separator
4–11	space, `( ) * ** + - /`	punctuation and operators (`**` is one token)
12–21	`0 ... 9`	digits; the number 15 is two tokens
22	`O(x**6)`	the whole error term is one token
23–39	`a b c d f g h m n x, exp sin cos tan sinh cosh tanh`	symbols and function names

2.2 One sequence, laid out

The first sequence in the data file has the prompt `cosh(a*x)**3 + tanh(b*x)`. Its layout by position is shown in Table 3.

Table 3: The first sequence of the data file, by position.

Positions	Tokens	Codes
0	`<sos>`	1
1–15	`cosh ( a * x ) ** 3 + tanh ( b * x )`	`27 5 23 7 39 6 8 15 9 38 5 24 7 39 6`
16	`<sep>`	3
17–70	`1 + b * x + ... + O(x**6)`	`13 9 24 ... 22`
71	`<eos>`	2
72–127	`<pad> ... <pad>`	`0 ... 0`

A sequence therefore has the general form

$$\boldsymbol{X} = \texttt{<sos>},\ p_1, \ldots, p_m,\ \texttt{<sep>},\ t_1, \ldots, t_k,\ \texttt{<eos>},\ \texttt{<pad>}, \ldots, \texttt{<pad>}, \tag{1}$$

with $m = 15$ prompt tokens and $k = 54$ target tokens in this example. Two details matter later.

- The <eos> token immediately follows the last target token (position 71 here), and the padding comes after it. The model therefore learns to end its output with <eos>.
- After filtering by length, the file holds 12,956 sequences of mean unpadded length 68.2 tokens (standard deviation 20.6), so on average about half of each row is padding.

An exhaustive search over outputs is hopeless: with $|\mathbb{V}| = 40$ and $k \approx 54$ there are about $40^{54} \approx 10^{86}$ candidate strings. The model must therefore be used with a heuristic, here greedy decoding (Section 6).

3 The architecture at a glance

Figure 1 shows the whole model. Read it top to bottom. The shape $[B, L, 72]$ is preserved from the embedding to the last layer normalization, so every position carries its own 72-vector through the network. The two elbows on the left are the residual (skip) paths, and the mask built from the token codes enters only the attention block, the one place where positions exchange information.

4 The components, one at a time

Each subsection gives the mathematics, the tensor shapes, and the class in the notebook that implements it, following the data through the model.

4.1 Embedding: tokens and positions become points (`Decoder`)

Two learned lookup tables do the embedding: a token table E_{tok} of shape 40×72 and a position table E_{pos} of shape 128×72 (both `nn.Embedding`). The token x_i at position i becomes the vector

$$v_i = \nu\, E_{\text{tok}}[x_i] + E_{\text{pos}}[i] \in \mathbb{R}^{72}, \tag{2}$$

where ν is a trainable scalar initialized to $\sqrt{72} \approx 8.49$. Dropout is applied to v, and the result has shape $[B, L, 72]$: the sequence is now a cloud of L points in $\mathbb{R}^{72}$. In the notebook's geometric language, the token vector t and the position vector z span a line, and $v = \nu t + z$ is a point on it.

The position term is essential. Attention, defined below, treats its input as an unordered set: permute the tokens and the outputs are permuted in the same way. Without E_{pos} the model could not tell `x**2/3` from `x**3/2`. The positions here are *learned* vectors, not the fixed sinusoids of Vaswani *et al.*

4.2 Masks: who may look at whom (`TinyLM.make_mask`)

The mask is a boolean array M of shape $[B, 1, L, L]$; the singleton axis lets one mask serve all eight heads by broadcasting. Its entry for query position i and key position j is

$$M_{ij} = \underbrace{[\, j \le i \,]}_{\text{causal (`torch.tril`)}} \wedge \underbrace{[\, x_j \neq \texttt{<pad>} \,]}_{\text{pad}}. \tag{3}$$

For the five-token sequence <sos>, p, <sep>, t_1, <eos> the causal part is lower triangular (Figure 2): row i lists the positions that token i may attend to.

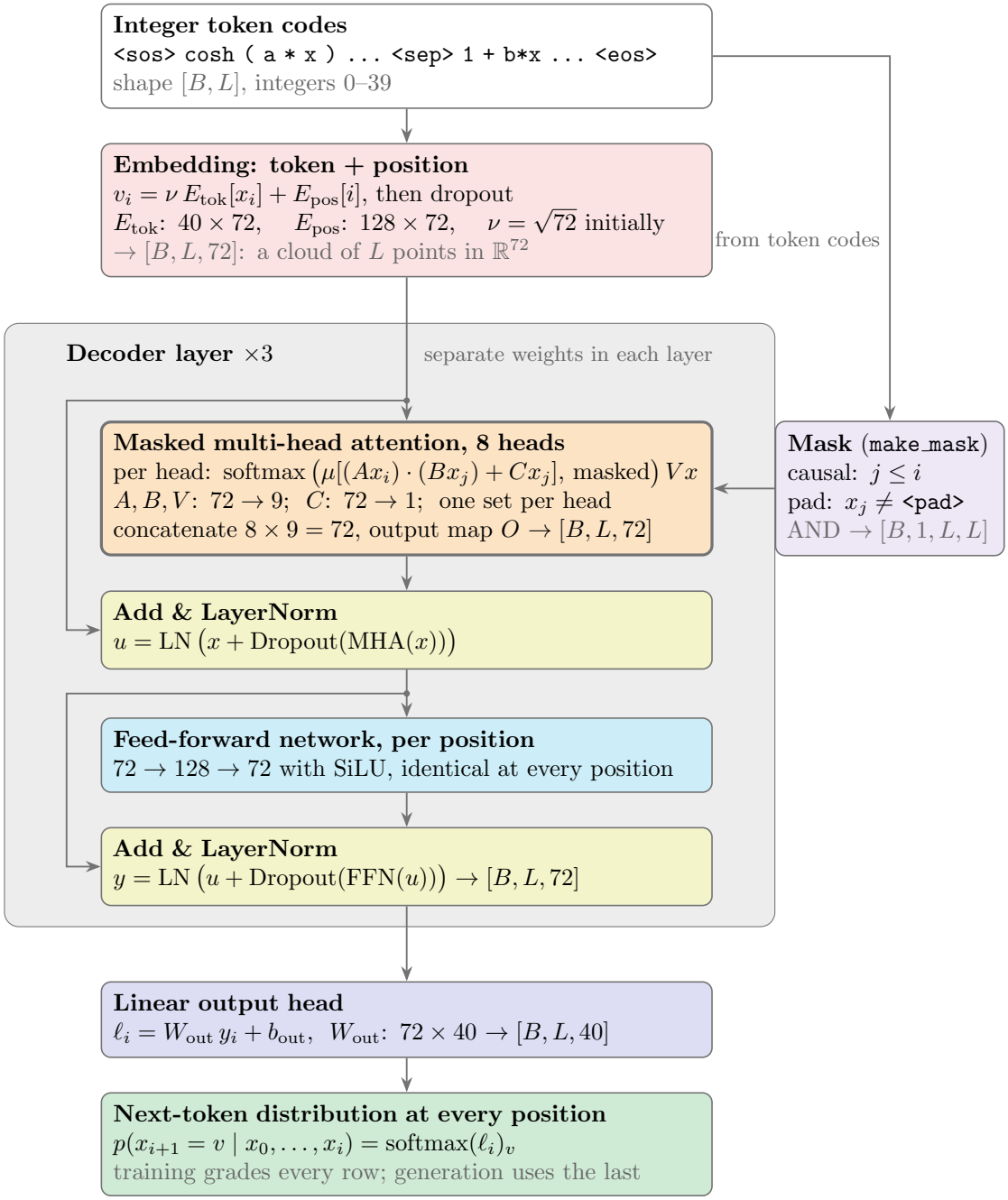


Figure 1: The tiny language model. Attention (orange) is the only step in which positions exchange information. Shapes are given for batch size B and sequence length L.



Figure 2: The causal mask for a five-token sequence. Row i marks the tokens that token i may attend to: itself and those before it.

Wherever $M_{ij} = 0$ the attention score is overwritten with -10^{10} before the softmax, so its weight becomes exactly zero. The pad mask removes *columns* (pads cannot be looked at); it does not remove rows. With `PROMPT_MASK = True` the prompt block would be fully visible to itself (a "prefix LM"); the notebook keeps it `False`, so the prompt, too, is read strictly left to right.

4.3 Attention: the (A, B, C) parametrization (`AttentionHead`, `MultiHeadAttention`)

This is the heart of the model.

The textbook form. With $Q = xA_Q^T + b_Q$, $K = xA_K^T + b_K$ and $V = xA_V^T + b_V$, attention is $\text{softmax}(\mu QK^T)V$. Expanding one score gives

$$Q_i K_j^T = x_i A_Q^T A_K \, x_j^T + x_i A_Q^T b_K^T + b_Q A_K \, x_j^T + b_Q b_K^T. \tag{4}$$

The symmetry of the softmax. The softmax acts along the key index j and is unchanged when the same constant is added to every entry of a row. The second and fourth terms do not depend on j, so they cancel: the key bias b_K never affects an attention weight. The third term survives, but it is a single fixed row vector, $C \equiv b_Q A_K$, dotted into each x_j, so it is given its own parameter.

The result. Writing $A \equiv A_Q$ and $B \equiv A_K$ (both without bias), head ℓ computes, for query position i,

$$S_{ij}^{(\ell)} = \mu_\ell \big[(A_\ell x_i) \cdot (B_\ell x_j) + C_\ell x_j \big], \tag{5}$$

$$w_{ij}^{(\ell)} = \frac{M_{ij} \, e^{S_{ij}^{(\ell)}}}{\sum_{j'} M_{ij'} \, e^{S_{ij'}^{(\ell)}}}, \tag{6}$$

$$\text{head}_\ell(x)_i = \sum_{j \le i} w_{ij}^{(\ell)} \left(V_\ell x_j + b_\ell \right), \tag{7}$$

with A_ℓ, B_ℓ, V_ℓ each 9×72, C_ℓ of size 1×72, $b_\ell \in \mathbb{R}^9$, and μ_ℓ a trainable scale initialized to $1/\sqrt{9} = 1/3$.

Every textbook head is an (A, B, C) head: take $C_\ell = b_{Q,\ell} A_{K,\ell}$. The converse is not quite true. In the textbook form that linear term is confined to the row space of $A_{K,\ell}$, of dimension at most 9, whereas C_ℓ may be any vector in $\mathbb{R}^{72}$. The (A, B, C) family is therefore slightly larger, and it no longer carries the key bias, which cannot affect the output. A trained $Q/K/V$ model can be converted into an (A, B, C) model exactly, which is how the checkpoint in `runs/TLM_ABC/` was made. The three pieces have simple readings.

- **A and B: a learned similarity.** $(A_\ell x_i) \cdot (B_\ell x_j) = x_i^T W_\ell x_j$ with $W_\ell = A_\ell^T B_\ell$, a 72×72 bilinear form of rank at most 9. Only W_ℓ is identifiable, so "query" and "key" are just names for its two low-rank factors.
- **C: a prior on each key.** $C_\ell x_j$ depends on token j alone. It says how attractive token j is to every query, whatever the query is.
- **The softmax: a convex combination.** Each row $w_{i\cdot}$ is non-negative and sums to one, so it is a point on the probability simplex. The head's output for token i therefore lies in the convex hull of the value vectors of tokens $0, \ldots, i$, and of nothing later.

The scale μ keeps the scores of order one. If the nine components of $A_\ell x$ and $B_\ell x$ have roughly unit variance, their dot product has variance about 9, and multiplying by $1/3$ prevents the softmax from saturating early in training.

Many heads. The eight heads act in parallel on the same input. Their 9-dimensional outputs are concatenated into one 72-vector and mixed by an output map O (72×72 plus bias):

$$\mathrm{MHA}(x)_i = O\big[\,\mathrm{head}_1(x)_i \,\|\, \cdots \,\|\, \mathrm{head}_8(x)_i\big] + b_O. \tag{8}$$

One head offers one rank-9 notion of similarity; eight heads let a token gather eight different kinds of context at once.

Two implementations, one function. `MultiHeadAttentionPerHead` builds each head as its own `AttentionHead` object and loops over them, so each head's parameters are visible in the code. `MultiHeadAttention`, the version the model uses, stacks the eight heads into single 72×72 layers and splits the result by reshaping. For equal weights the outputs are identical; the fused version is faster because a few large matrix products use a GPU far better than many small ones. Table 4 traces its shapes.

Table 4: Shape bookkeeping in `MultiHeadAttention.forward`.

Step	Operation	Shape
1	Ax, Bx, Vx	$[B, L, 72]$
2–3	split into heads and permute	$[B, 8, L, 9]$
4	$S = Ax\,(Bx)^T$	$[B, 8, L, L]$
5	add $C(x)$ (permuted to $[B, 8, 1, L]$); multiply by μ (shape $[8, 1, 1]$)	$[B, 8, L, L]$
6	mask with -10^{10}, softmax over keys, dropout	$[B, 8, L, L]$
7	WVx, then permute and merge heads	$[B, L, 72]$
8	output layer O	$[B, L, 72]$

A remark on the value bias. Within one head the value bias b_ℓ is not removed by the softmax: because each row of weights sums to one, it is carried through as a constant shift of the head's output. After concatenation, however, that shift becomes the constant vector $O[b_1 \| \cdots \| b_8]$, which the output bias b_O can absorb. At the level of the multi-head layer the value bias is therefore redundant (216 of the 136,169 parameters); it is kept to stay close to the standard form.

4.4 Feed-forward network (`Feedforward`)

After attention has mixed information *between* tokens, a small two-layer network processes each token *on its own*, with the same weights at every position:

$$\mathrm{FFN}(u) = W_2\,\mathrm{Dropout}\big(\mathrm{SiLU}(W_1 u + b_1)\big) + b_2, \qquad \mathrm{SiLU}(z) = z\,\sigma(z), \tag{9}$$

with $W_1 : \mathbb{R}^{72} \to \mathbb{R}^{128}$ and $W_2 : \mathbb{R}^{128} \to \mathbb{R}^{72}$. The dichotomy is worth stating plainly: attention is the *only* place where tokens communicate; the feed-forward network is where each token processes what it has gathered.

4.5 Residual connections and layer normalization (`DecoderLayer`)

A decoder layer wraps each of its two sub-layers in a residual connection followed by layer normalization:

$$u = \mathrm{LN}_1\left(x + \mathrm{Dropout}(\mathrm{MHA}(x))\right), \qquad y = \mathrm{LN}_2\left(u + \mathrm{Dropout}(\mathrm{FFN}(u))\right). \tag{10}$$

In the residual form each sub-layer learns a *correction* to its input rather than a replacement, which is what makes stacks of layers trainable. Layer normalization standardizes the 72 components of each token separately,

$$\mathrm{LN}(z) = \gamma \odot \frac{z - \mathrm{mean}(z)}{\sqrt{\mathrm{var}(z) + 10^{-5}}} + \beta, \qquad \gamma, \beta \in \mathbb{R}^{72} \text{ learned}. \tag{11}$$

Placing the normalization after the addition ("post-LN") follows the original 2017 paper. A layer maps $[B, L, 72]$ to $[B, L, 72]$, so identical layers can be stacked.

4.6 The output head (`Decoder.linear`)

A final linear map turns each 72-vector y_i into 40 logits, one per vocabulary token:

$$\ell_i = W_{\text{out}}\, y_i + b_{\text{out}} \in \mathbb{R}^{40}, \qquad p(x_{i+1} = v \mid x_0, \ldots, x_i) = \frac{e^{\ell_{i,v}}}{\sum_{u \in \mathbb{V}} e^{\ell_{i,u}}}. \tag{12}$$

The output has shape $[B, L, 40]$. Because of the causal mask, row i depends only on $x_0, \ldots, x_i$, so every row is a legitimate next-token prediction for its own prefix. One forward pass therefore yields L predictions, a fact that both training and generation exploit.

4.7 Where the parameters live

The three decoder layers hold 89% of the parameters.

Table 5: Parameter budget. The total matches the count printed by the notebook.

Component	Parameters
Token embedding, 40×72	2,880
Position embedding, 128×72	9,216
Embedding scale ν	1
One layer: attention (A, B: 5,184 each; C: 576; V, O: 5,256 each; μ: 8)	21,464
One layer: feed-forward ($9{,}344 + 9{,}288$)	18,632
One layer: two LayerNorms	288
Three decoder layers ($3 \times 40{,}384$)	**121,152**
Output head, $72 \times 40 + 40$	2,920
Total	**136,169**

5 Training: 127 predictions from one forward pass

During training the model is fed the *true* sequence and, thanks to the causal mask, predicts the next token at every position simultaneously. Feeding the ground truth rather than the model's own guesses is called **teacher forcing**. It is what makes transformers much cheaper to train than recurrent networks, which must step through a sequence one token at a time.

5.1 Aligning predictions with targets (`TLMObjective.forward`)

Logit row i predicts token $i + 1$, so the two tensors are shifted by one before they are compared: `logits[:, :-1, :]` drops the last row, because there is no token after the last position to predict, and `sequence[:, 1:]` drops `<sos>`, because nothing predicts it. Both then have 127 positions, and the loss at each is the cross entropy

$$\text{loss}_i = -\log \frac{e^{\ell_{i,x_{i+1}}}}{\sum_{u \in \mathbb{V}} e^{\ell_{i,u}}}. \tag{13}$$

5.2 Which predictions are graded (`TinyLM.make_loss_mask`)

The model is graded only on the answer, not on the prompt. A loss mask keeps the prediction made at position i when i is at or after `<sep>` and its *target* x_{i+1} is a real token, that is, an answer token or the `<eos>` that ends the answer. The empirical risk is the mean loss over the kept predictions:

$$m_i = [\, i \geq i_{\text{sep}} \,] \wedge [\, x_{i+1} \neq \texttt{<pad>} \,], \qquad R = \frac{\sum_{b,i} m_{b,i} \, \text{loss}_{b,i}}{\sum_{b,i} m_{b,i}}. \tag{14}$$

For the example of Table 3 this gives Table 6.

The model is thus graded on 55 predictions: the 54 answer tokens, plus the `<eos>` that ends the answer. Note that the mask is defined by the *target*. A mask defined by the input alone (keep i if $x_i \neq$ `<pad>`) would also grade the meaningless prediction made at `<eos>`, whose target is `<pad>`.

5.3 Optimization

- Adam, with the gradient norm clipped at 1.

Table 6: Which predictions enter the loss for the example sequence.

Input position	Input token	Target token	Graded?
0–15	`<sos>` and prompt	prompt and `<sep>`	no
16	`<sep>`	$t_1 =$ `1`	yes
17–69	$t_1, \ldots, t_{53}$	$t_2, \ldots, t_{54}$	yes
70	$t_{54} =$ `0(x**6)`	`<eos>`	yes
71	`<eos>`	`<pad>`	no
72–126	`<pad>`	`<pad>`	no

- 8×10^5 iterations of batch 32, so each of the 12,000 training sequences is seen about 2,100 times.
- Learning rate 1.6×10^{-3}, multiplied by $\gamma = 0.831$ every 50,000 iterations, ending at 1.0×10^{-4}.
- Dropout ($p = 0.1$) is active in `train()` mode and switched off in `eval()` mode, which is used for monitoring and generation.
- Every 100 iterations the monitor evaluates the risk on 156 training and 156 validation sequences and saves the parameters when the validation risk improves.

6 Autoregressive generation

At inference time the model is run once per output token: the last row of logits picks the next token, which is appended to the input and fed back in (Figure 3). Nothing in the network changes between training and generation; only the way it is called does.

6.1 The algorithm (`greedy_generate`)

1. **Start** with the delimited prompt $s =$ `<sos>`$, p_1, \ldots, p_m,$ `<sep>`, of shape $[1, m{+}2]$; `seqdata.split` cuts it from a test sequence.
2. **Run the model** on s in `eval()` mode, so that dropout is off, with the same causal mask as in training. The output has shape $[1, n, 40]$ for the current length n.
3. **Keep only the last row** ℓ_{n-1}. By the causal mask it depends on all of s and on nothing else, so it defines the distribution of the next token given everything so far.
4. **Choose greedily**: $y = \arg\max_v \ell_{n-1,v}$.
5. **Stop or append.** If y is `<eos>`, stop. Otherwise append y, so that $n \to n + 1$, and return to step 2. The loop runs at most $128 - (m + 2)$ times. (The code also stops on `<pad>`, a safeguard for models trained before `<eos>` was placed directly after the answer.)

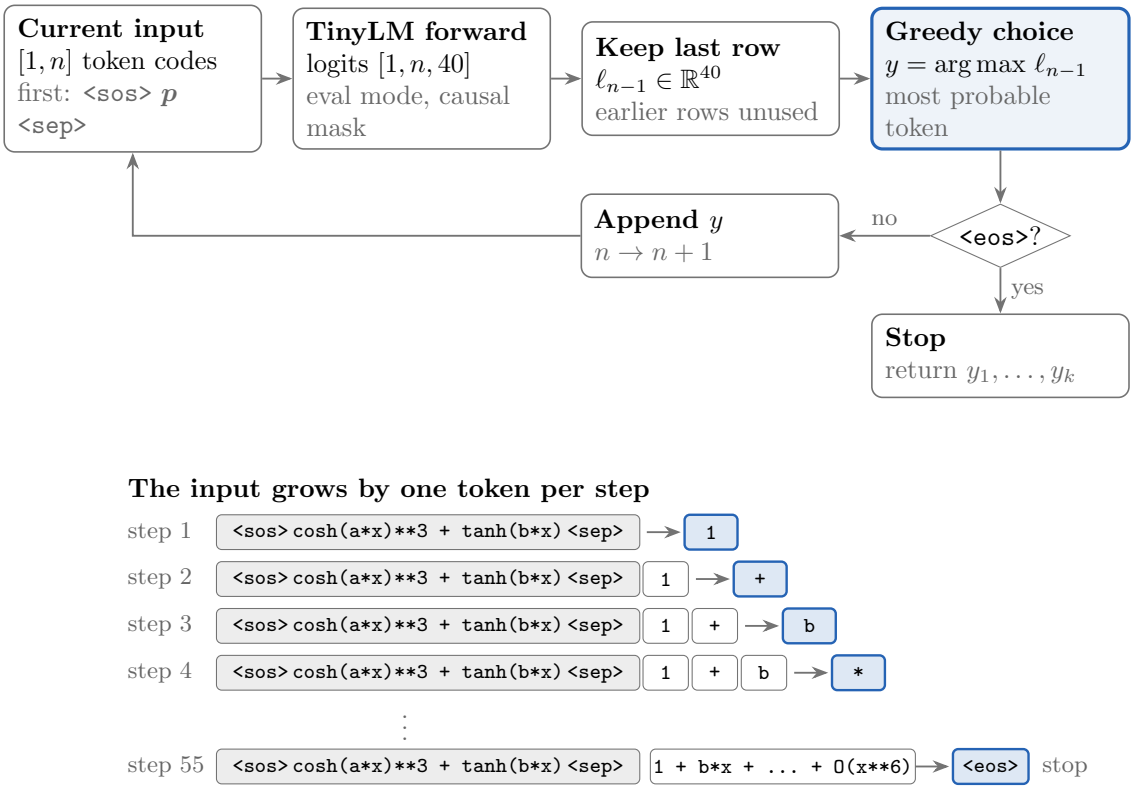


Figure 3: Greedy autoregressive generation (`greedy_generate`). Top: one iteration of the loop. Bottom: the input for the example of Table 3; shaded is the delimited prompt, highlighted the token predicted at each step, which becomes input at the next step.

6.2 What the loop computes

The network defines the probability of a complete answer as a product of next-token factors, each conditioned on the prompt and on the answer so far:

$$p(t_1, \ldots, t_k \mid \boldsymbol{p}) = \prod_{i=1}^{k} p\big(t_i \,\big|\, \texttt{<sos>}, \boldsymbol{p}, \texttt{<sep>}, t_1, \ldots, t_{i-1}\big). \tag{15}$$

Greedy decoding maximizes each factor in turn, which does not guarantee the largest product: an early, locally best token can lead to a worse sequence overall. Beam search (Section 6.3) explores more of the space. For this deterministic task greedy decoding already succeeds on $99.3\% \pm 0.3\%$ of the 800 test functions, where the uncertainty is the binomial $\sqrt{F(1-F)/N}$.

6.3 Beam search (`beam_generate`)

Beam search keeps the W most probable partial answers, called *beams*, instead of only one; the notebook uses $W = 5$. A beam's score is the log-probability of its partial answer, $\log P(t_1, \ldots, t_i \mid \boldsymbol{p}) = \sum_{r \le i} \log p(t_r \mid \ldots)$, so extending a beam by the token v simply adds $\log p(v \mid \text{beam})$ to its score. Figure 4 shows one step of the loop and a small example with $W = 2$.

The loop in the code is as follows.

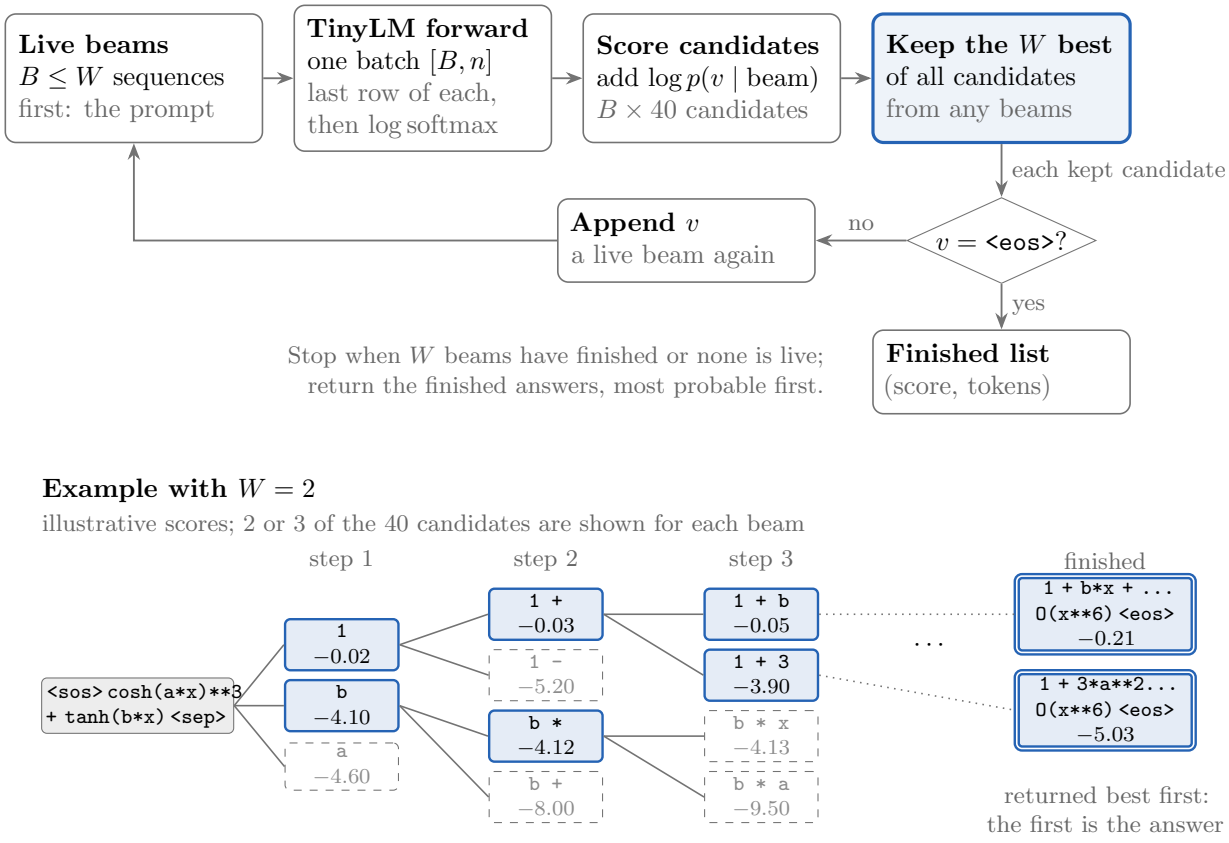


Figure 4: Beam search (`beam_generate`). Top: one step of the loop. Bottom: an example with $W = 2$ for the prompt of Table 3. Highlighted beams are the W best of *all* candidates at that step; dashed ones are dropped. At step 3 both survivors descend from `1 +`, while `b *` leaves no child. Numbers are log-probabilities of the partial answers and are illustrative.

1. **Start** with a single live beam, the delimited prompt, with score 0, and an empty list of finished answers.
2. **Run the model once on all live beams.** They all have the same length n, so they form one batch of shape $[B, n]$ with $B \le W$. The last row of logits of each beam gives, through a log-softmax, $\log p(v \mid \text{beam})$ for all 40 tokens v.
3. **Score every candidate**, $S(\text{beam}, v) = \log P(\text{beam}) + \log p(v \mid \text{beam})$: $B \times 40$ numbers.
4. **Keep the W largest** of these, whichever beams they come from. A strong beam may keep several children and a weak one none.
5. **Sort the kept candidates.** One whose token v is `<eos>` is finished: its answer (without `<eos>`) and score go to the finished list. The others are extended by v and become the live beams of the next step.
6. **Stop** when the finished list holds W answers, when no beam is live, or at the length limit. Return the finished answers sorted by score, most probable first.

Four properties follow directly from the code.

- $W = 1$ **is greedy decoding.** With one beam, the best candidate is the arg max of

$\log p(v \mid \text{beam})$, the greedy token.

- **Cost.** Each step is one forward pass on a batch of at most W sequences: roughly W times the arithmetic of greedy decoding, for the same number of steps.
- **A pruned search.** A candidate dropped at one step is gone for good, so beam search is still a heuristic. It usually finds a more probable answer than greedy decoding, but neither is guaranteed to find the most probable one.
- **No length normalization.** Every token adds $\log p \leq 0$ to the score, so, among finished answers, shorter ones are favored. Many implementations divide the score by a power of the length. Here the length of the answer is fixed by the function, so the effect is small.

Table 7: Training and generation compared.

	Training	Generation
Input	the true sequence (teacher forcing)	the model's own previous outputs
Forward passes per sequence	1	one per output token (55 for the example)
Logit rows used	all 127, masked to the answer	only the last
Dropout	on	off
Causal mask	yes	yes

6.4 Why earlier positions need not be recomputed

Because of the causal mask, appending a token at the end does not change the computation for any earlier position: rows $0, \ldots, n-1$ at step $n+1$ are identical to rows $0, \ldots, n-1$ at step n. `greedy_generate` recomputes them anyway, so each step repeats all the earlier work. Caching each layer's Bx, Cx and Vx for the past tokens (a key/value cache) lets each step compute only the new row, which is how production language models generate quickly.

7 Beyond this model

The tiny language model follows the 2017 design closely. Current language models differ from it in a few ways that are easy to state.

- **Post-LN versus pre-LN.** This model normalizes after each residual addition, as in the 2017 paper. Most current language models normalize *before* each sub-layer and add a final layer normalization, which trains more stably in deep stacks.
- **Positions.** Learned absolute position vectors cap the length at 128. Modern models typically use relative schemes such as rotary embeddings (RoPE).
- **Key/value cache.** Production models cache past keys and values, so that each generation step computes only one new row (Section 6).
- **Tied embeddings.** Here the output map W_{out} is separate from the token table E_{tok}; many language models tie them.
- **The end of the output is learned.** `<eos>` is just another token, which the model learns to emit because it always follows the answer in the training data. Had the padding been placed before `<eos>`, the model would have learned to end its output with `<pad>` instead.

References

[1] A. Vaswani *et al.*, "Attention is all you need," *Advances in Neural Information Processing Systems* **30** (2017). NeurIPS proceedings.

[2] A. Rush *et al.*, "The Annotated Transformer." nlp.seas.harvard.edu/annotated-transformer.

[3] J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer normalization," arXiv:1607.06450 (2016).